CSE 4304-Data Structures Lab. Winter 2022
Date: November 01, 2022.
Target Group: All
Topic: Hashing-2

Instructions:

- Task naming format: fullID_T01L11_2A.c/CPP
- If you find any issues in the problem description/test cases, comment in the google classroom.
- If you find any tricky test case that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to recall the solution in the future easily.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	Tasks
1B	1 2 3
2B	1 3 4
2A	1 3 4
1A	1 3 4

Task-1

Implement the following Collision handling techniques:

1. Linear Probing
2. Quadratic Probing
3. Double Hashing

The first input line should be (choice, N, Q), where 'choice' can be 1/2/3 corresponding to linear/quadratic/double hashing. N represents the size of the HashTable. Q represents the number of queries.
Then there will be Q numbers given as input.

Hash(x) = (x % TableSize)
Choose a suitable hash function for double hashing in addition to the function as mentioned above (as it needs two)

Sample Input	**Sample Output**
1 10 8 35 45 73 36 5 24 13 **99**	 Inserted : Index-5 (L.F=.1) Collision: Index-5 Inserted : Index-6 (L.F=.2) Inserted : Index-3 (L.F=.3) Collision: Index-6 Inserted : Index-**7** (L.F=.4) Collision: Index-5 Collision: Index-6 **Collision: Index-7** Inserted : Index-**8** (L.F=.5) Inserted : Index-4 (L.F=.6) Collision: Index-3 Collision: Index-4 Collision: Index-5 Collision: Index-6 Collision: Index-7 **Collision: Index-8** **Input Abandoned** **Inserted : Index-9 (L.F=.7)**
2 8 7 67 15 86 63 47	 Inserted : Index-3 (L.F = 0.125) Inserted : Index-7 (L.F = 0.25) Inserted : Index-6 (L.F = 0.375) Collision: Index-7 Inserted : Index-0 (L.F = 0.5) Collision: Index-7 Collision: Index-0 Collision: Index-3 Collision: Index-0 Collision: Index-7

33 8	Collision: Index-0 Input Abandoned Inserted : Index-1 (L.F = 0.625) Collision: Index-0 Collision: Index-1 Inserted : Index-4 (L.F = 0.75)
3 15 11 94 46 61 29 85 77 46 63 67 93 61	 Inserted : Index-4 (L.F = 0.0666667) Inserted : Index-1 (L.F = 0.133333) Collision: Index-1 Inserted : Index-3 (L.F = 0.2) Inserted : Index-14 (L.F = 0.266667) Inserted : Index-10 (L.F = 0.333333) Inserted : Index-2 (L.F = 0.4) Collision: Index-1 Collision: Index-4 Inserted : Index-7 (L.F = 0.466667) Collision: Index-3 Collision: Index-10 Collision: Index-2 Inserted : Index-9 (L.F = 0.533333) Collision: Index-7 Collision: Index-10 Inserted : Index-13 (L.F = 0.6) Collision: Index-3 Inserted : Index-8 (L.F = 0.666667) Collision: Index-1 Collision: Index-3 Inserted : Index-5 (L.F = 0.733333)

Note:

- L.F means Load Factor.
- If you fail to insert the number within six attempts, abandon that number.

(Please test your program for different TableSize and different sets of numbers)

Task-2

Implement the **‘Rabin-Karp String Matching Algorithm’** using the concept of the **rolling-hash function**. Test your program for different test cases. Make sure you understand how this algorithm is improving the traditional approach.

Task-3

The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'. For example, "ACGAATTCCG" is a DNA sequence. When studying DNA, it is useful to identify repeated sequences within the DNA.

Given a string ‘s’ that represents a DNA sequence, return all the 10-letter-long sequences that occur more than once in a DNA molecule. You may return the answer in any order.

Constraints:

- 1 <= s.length <= 10^5
- s[i] is either 'A', 'C', 'G', or 'T'.

Sample Input	**Sample Output**
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC, CCCCCAAAAA
AAAAAAAAAAAAAAAA	AAAAAAAAAA

Note: Use the Rabin-Karp string matching algorithm to calculate the hash value. May use unordered maps for other purposes.

Task 4:

Given two strings s1 and s2, your task is to find the **longest common substring.**

Input	**Output**
abcdxyz xyzabcdbcd	abcd
zxabcdezyab yzabcdezx	abcdez
abcdefgh hgfedcba	a (other solutions are possible like b/c/d/e/f/g/h)
aaaabbbbcccc aaaaaaaaaaabbbbbbbbbbcccccccccc	aaaabbbb (alternative: bbbbcccc)
abcdefg ijklmno	Null

Note:

- Provide **O(mlogN)** solution utilizing the concept of Hashing. (m and N represent the length of the smaller and longer strings)
- If there are alternative choices, show any of them.